

AI-BASED ONLINE EXAM PROCTORING SYSTEM

Dissertation submitted to

BHARATHIDASAN UNIVERSITY, TIRUCHIRAPPALLI
in partial fulfillment of the requirements for the award of the Degree of



BACHELOR OF SCIENCE IN COMPUTER SCIENCE

submitted by

Name	Register No.
J. PRINSON JOSE	CB22S611365
P. SWETHA	CB22S611377
A. VETHILRAJ	CB22S611379
R. VISHWA	CB22S611380

Under the Guidance of

Mrs. A. SUDHA, M.Sc., M.Phil., B.Ed.,
Lecturer, Department of Computer Science



DEPARTMENT OF COMPUTER SCIENCE

MANNAI RAJAGOPALASAMY GOVT. ARTS COLLEGE

MANNARGUDI-614 001.

MARCH-2025



DEPARTMENT OF COMPUTER SCIENCE
M.R. GOVERNMENT ARTS COLLEGE
Mannargudi-614001

Mrs. A. SUDHA, M.Sc., M.Phil., B.Ed.,
Lecturer, Department of Computer Science

Date:

CERTIFICATE

This is to certify that the dissertation entitled “**AI-BASED ONLINE EXAM PROCTORING SYSTEM**” submitted by **J. PRINSON JOSE (Reg.No.CB22S611365), P.SWETHA (Reg.No.CB22S611377), A.VETHILRAJ (Reg.No.CB22S611379), R.VISHWA (Reg.No.CB22S611380)** in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE IN COMPUTER SCIENCE**, M.R. Government Arts College, Mannargudi 614001, is a bonafide work done by them under my guidance and supervision during the academic year 2024-2025. This dissertation has not previously formed the basis for the award of any degree of fellowship.

Signature of the HOD

Signature of the Guide

Submitted for the Viva-voce examination held on _____

Examiners

1.

2.

ACKNOWLEDGEMENT

We express our gratefulness to “**MANNAI RAJAGOPALASAMY GOVERNMENT ARTS COLLEGE, MANNARGUDI** ” for the extreme service it offers us. At the first outset, We would like to express our sincere thanks to our Principal **Dr. D. RAJENDRAN, M.A., M. Phil., Ph.D.**, for providing lab facilities to success in our project.

We extend our thanks to our Head of the Department **Mr. S. V. NETHAJI, M.C.A. M.Phil.**, for his valuable advice and encouragement to do this project successfully.

We extend our thanks to our Project Guide **Mrs. A. SUDHA, M.Sc., M.Phil., B.Ed.**, for her valuable advice and encouragement to do this project.

Words are not enough to express our gratitude to our beloved parents for supporting us to do this project and also their words of encouragement during this project.

Finally we wish to extend deep sense of thanks to our friends for their untiring and kind help in the neat submission of this project.

AI-BASED ONLINE EXAM PROCTORING SYSTEM

CONTENTS

S.NO	TITLE	PAGE.NO
1	INTRODUCTION 1.1 ABSTRACT	1
2	SYSTEM STUDY AND ANALYSIS 2.1 PROBLEM DESCRIPTION 2.2 EXISTING SYSTEM 2.3 PROPOSED SYSTEM	 2 3 4
3	SYSTEM ENVIRONMENT 3.1 SYSTEM CONFIGURATION 3.2 ABOUT WINDOWS 10 3.3 ABOUT VS CODE 3.4 ABOUT PYTHON 3.10 3.5 ABOUT OPENCV, MEDIAPIPE, PYAUDIO, FACE RECOGNITION, MATPLOTLIB	 5 6 8 9 11
4	SYSTEM DESCRIPTION 4.1 SYSTEM FLOW DIAGRAM 4.2 IMPLEMENT METHODOLOGY	 14 15
5	SYSTEM DESIGN AND DEVELOPMENT	17
6	SYSTEM TESTING AND MAINTENANCE	19
7	SAMPLE CODING	21
8	SAMPLE SCREEN DISPLAY	25
9	CONCLUSION	28
10	REFERENCES	29

1.1 ABSTRACT

With the rise of online education and remote assessments, ensuring the integrity of online exams has become a critical challenge. Traditional proctoring methods, such as manual invigilation or webcam monitoring, often suffer from scalability issues, privacy concerns, and inefficiencies. To address these challenges, this project introduces an **AI-based online Exam Proctoring System**, which leverages artificial intelligence and computer vision techniques to monitor and analyze student behaviour during examinations.

The system utilizes real-time face detection, head pose estimation, and eye movement tracking to identify suspicious activities such as looking away frequently, multiple faces in the frame, or potential cheating behaviors. Additionally, machine learning algorithms analyze audio and visual inputs to detect anomalies, ensuring a fair examination environment. The system records and processes exam sessions, generating automated reports that highlight potential violations for further review by administrators.

By integrating AI-powered monitoring, this proctoring system enhances the security and credibility of online assessments while reducing the need for constant human supervision. This project demonstrates how artificial intelligence can be effectively applied to remote exam proctoring, providing an efficient, scalable, and automated solution for modern educational institutions and certification bodies.

2.1 PROBLEM DESCRIPTION

With the increasing adoption of online exams in education and certification, ensuring the security and fairness of these assessments has become a significant challenge. Traditional proctoring methods, such as manual invigilation and webcam monitoring, have several limitations that hinder their effectiveness. Human invigilators are required to oversee multiple candidates simultaneously, which makes it difficult to detect subtle or sophisticated cheating techniques. Additionally, manual proctoring is time-consuming and does not scale efficiently for large-scale online exams, making it impractical for institutions conducting assessments for thousands of candidates.

One of the major concerns with traditional proctoring is the invasion of privacy. Continuous human monitoring through webcams creates discomfort for examinees, raising ethical and legal concerns regarding data security and surveillance. Many candidates feel uneasy knowing that they are being watched at all times, which may impact their performance and overall exam experience. Moreover, reviewing recorded exam sessions manually is labour-intensive and impractical for exam administrators, leading to inefficiencies in the evaluation process.

To overcome these challenges, an **AI-Based Online Exam Proctoring System** provides a modern and automated approach to monitoring online assessments. By leveraging artificial intelligence and computer vision, the system analyzes various behavioral and environmental factors in real time to detect suspicious activities. Facial expressions, head movements, and gaze direction are monitored to identify potential cheating attempts. Additionally, the system can recognize unauthorized individuals in the testing environment and detect suspicious object usage, further enhancing exam security.

This AI-driven approach significantly reduces human intervention, streamlines the proctoring process, and ensures a fair assessment environment for all candidates. By automating monitoring and analysis, the system provides a scalable and efficient solution for online exam security while addressing privacy concerns. Institutions and certification bodies can rely on this advanced technology to maintain the integrity of their exams without the need for continuous human supervision, making online assessments more reliable and trustworthy.

2.2 EXISTING SYSTEM

Traditional online exam proctoring methods rely on manual invigilation or basic webcam monitoring. In many institutions and certification bodies, exams are monitored through live video feeds, where human proctors observe candidates in real-time. This approach is time-consuming, labour-intensive, and difficult to scale for large numbers of students. Additionally, relying solely on human proctors increases the risk of human error and subjective decision-making, leading to inconsistencies in exam monitoring.

To address some of these challenges, automated proctoring systems have been introduced, including screen recording and basic AI-based face detection. However, these systems have limitations. Simple face detection fails to identify sophisticated cheating methods, such as using secondary devices, multiple individuals in the room, or looking at unauthorized materials. Some systems also generate a high number of false positives, leading to unnecessary disruptions during exams. Privacy concerns further complicate traditional proctoring solutions, as continuous monitoring can make candidates uncomfortable and raise ethical concerns.

Due to these limitations, there is a growing need for a more intelligent, automated, and non-intrusive online exam monitoring solution. This has led to the development of AI-based proctoring systems that leverage machine learning and computer vision to ensure secure, scalable, and efficient exam supervision.

2.3 PROPOSED SYSTEM

The proposed system introduces an AI-Based Online Exam Proctoring System that automates the monitoring of online exams efficiently and accurately. This system utilizes artificial intelligence, computer vision, and machine learning to detect and analyze student behavior during examinations.

It continuously tracks facial expressions, head movements, and environmental factors to identify suspicious activities such as looking away frequently, multiple faces in the frame, or unauthorized device usage.

This system eliminates the need for constant manual invigilation, reducing human errors and making online exams more secure and scalable. Unlike traditional proctoring methods, AI-based monitoring ensures real-time detection of anomalies without requiring human intervention. Additionally, the system generates automated reports highlighting potential violations, allowing administrators to review flagged instances efficiently.

The AI-Based Online Exam Proctoring System is cost-effective, privacy-conscious, and highly scalable, making it an ideal solution for educational institutions, certification bodies, and organizations conducting remote assessments.

3.1 SYSTEM CONFIGURATION

HARDWARE REQUIREMENTS

- Processor : Dual-core processor 2.6 GHz
- RAM : 8GB
- Hard Disk : 239GB
- Keyboard : Standard keyboard
- Monitor : 15-inch color monitor
- Webcam : Basic VGA (Low-quality face detection)
- Microphone : Built-in microphone for audio analysis

SOFTWARE REQUIREMENTS

- Operating System : Windows 10
- Programming Language : Python 3.10
- IDE : VS Code
- Libraries : OpenCV, MediaPipe, PyAudio, Face Recognition, Matplotlib

3.2 ABOUT WINDOWS 10

Windows 10 is a widely used operating system developed by Microsoft, released on July 29, 2015. It is designed to provide a seamless and user-friendly computing experience for personal, business, and enterprise users. Windows 10 introduced several improvements over its predecessors, including a modern interface, enhanced security features, and better performance optimization. One of its key features is the Start Menu, which combines elements of both Windows 7 and Windows 8, providing users with a familiar yet enhanced navigation experience.

Windows 10 offers a range of built-in applications and productivity tools, such as Microsoft Edge, Cortana, and virtual desktops. Microsoft Edge is the default web browser, offering improved speed, security, and integration with Microsoft services. Cortana, the AI-powered virtual assistant, helps users with tasks such as setting reminders, searching files, and managing calendar events through voice commands. Virtual desktops allow users to organize and manage multiple workspaces efficiently, enhancing multitasking capabilities.

Another major advantage of Windows 10 is its strong focus on security and updates. With the introduction of Windows Defender Antivirus and frequent security patches, the operating system ensures protection against malware, ransomware, and other cyber threats. Windows Update provides automatic updates, ensuring that users always have the latest security enhancements and feature improvements. Additionally, Windows 10 includes BitLocker encryption, which enhances data security for business and enterprise users.

Windows 10 also introduced DirectX 12, a significant upgrade for gamers, improving graphics performance and reducing latency in high-end games. The Xbox Game Bar allows users to capture gameplay, stream content, and communicate with friends directly from their desktop. Additionally, the operating system features a more efficient file management system, a new Settings app that replaces the traditional Control Panel, and seamless integration with OneDrive for cloud storage.

Another feature that sets Windows 10 apart is its adaptability across different devices, including desktops, laptops, tablets, and hybrid devices. It features a Continuum mode that allows users to switch between tablet and desktop interfaces seamlessly, making it ideal for hybrid devices like the Microsoft Surface. The integration of touch and stylus support further enhances the versatility of the operating system, catering to creative professionals and students alike.

With the introduction of Windows Subsystem for Linux (WSL), Windows 10 enables developers to run Linux-based applications directly on their machines, bridging the gap between Windows and Linux environments. This feature is particularly useful for developers and IT professionals who require cross-platform compatibility.

Windows 10 also features Windows Ink, which enhances stylus support, allowing users to take handwritten notes, draw, and annotate documents easily. This is particularly useful for digital artists, students, and professionals who rely on pen input for creative tasks. The Microsoft Store provides access to a vast collection of applications, games, and utilities, making it easy for users to find and install software tailored to their needs.

Windows 10 includes various accessibility improvements, such as the Narrator screen reader, speech recognition, and magnifier tools, ensuring a more inclusive experience for users with disabilities. Microsoft has also implemented enterprise-specific functionalities like Windows Hello for biometric authentication and Azure Active Directory integration for secure login and remote work capabilities.

Windows 10 is designed to be a long-term, continually evolving operating system. Microsoft has implemented the "Windows as a Service" model, providing regular feature updates and improvements instead of releasing entirely new versions. With its continuous updates, comprehensive security, and wide range of features, Windows 10 remains one of the most reliable and versatile operating systems available today, catering to casual users, students, professionals, and enterprises across various industries.

3.3 ABOUT VS CODE

Visual Studio Code (VS Code) is a free, open-source code editor developed by Microsoft that is widely used for software development. It provides a lightweight yet powerful environment with support for multiple programming languages, including Python, JavaScript, C++, Java, and more. VS Code is known for its speed, flexibility, and extensive customization options, making it a preferred choice among developers for writing, debugging, and testing code efficiently.

One of the standout features of VS Code is its robust extension marketplace, which allows developers to add functionality such as syntax highlighting, debugging tools, and language support. The built-in IntelliSense feature enhances productivity by offering smart code completions and real-time error detection. Additionally, VS Code supports Git integration, enabling seamless version control and collaboration among teams.

VS Code is cross-platform and runs on Windows, Linux, and macOS, making it accessible to a wide range of users. Its lightweight nature combined with powerful features like a built-in terminal, live server support, and remote development capabilities make it an essential tool for modern developers. With continuous updates and a strong developer community, VS Code continues to evolve, providing an efficient and feature-rich environment for coding, testing, and debugging applications across various domains.

3.4 ABOUT PYTHON 3.10

Python 3.10, released on October 4, 2021, is one of the most significant updates in the Python programming language, introducing various improvements in performance, security, and syntax. As an interpreted, high-level, and dynamically typed language, Python is widely used in various fields, including web development, data science, artificial intelligence, and automation. With each iteration, Python continues to evolve, making coding more efficient and readable, and Python 3.10 is no exception.

One of the most notable features introduced in Python 3.10 is the structural pattern matching feature, which enhances the way developers write conditional statements. The "match" statement allows pattern matching similar to a switch-case structure found in other programming languages like C and JavaScript. This new feature enables more readable and expressive code, particularly useful in applications involving complex data structures, parsers, and decision trees.

Another improvement in Python 3.10 is better error messages and debugging support. Prior versions of Python provided error messages that were sometimes ambiguous or difficult to interpret, but Python 3.10 introduces more descriptive and precise error messages. For instance, it highlights specific portions of code where syntax errors occur, making debugging easier for developers, especially beginners.

Python 3.10 also includes enhancements to type hinting, making static type checking more robust. The introduction of "Parameter Specification Variables" and "Precise Types" allows for more accurate type annotations, helping developers write safer and more maintainable code. Type hints have become increasingly important for large-scale projects, where maintaining code readability and reducing runtime errors are essential. Additionally, Python 3.10 enhances the `zip()` function with a new `strict` parameter, which raises an error when input iterables have different lengths, preventing unintended behavior.

Performance improvements in Python 3.10 also make it a more efficient language. The interpreter has been optimized, resulting in faster execution times for certain operations. Python developers benefit from these improvements when working on performance-critical applications, such as data processing and machine learning tasks. The new optimizations in Python 3.10 include better garbage collection and improved memory management, reducing memory overhead and improving execution speed in multi-threaded applications.

Another significant addition is the introduction of the "parenthesized context managers" feature, which allows multiple context managers to be used more cleanly and efficiently. This enhancement simplifies resource management, especially when handling files, network connections, or database sessions. It improves the readability of the "with" statement and makes Python programs more maintainable. Moreover, Python 3.10 introduces new syntax changes, such as allowing union types to be written more succinctly using the `|` operator, making type annotations cleaner and easier to understand.

Python 3.10 also enhances security by improving hashing algorithms and introducing more secure implementations for certain built-in functions. With growing concerns over cybersecurity threats, these updates ensure that Python remains a safe and reliable language for developing secure applications. Additionally, improvements in memory safety features reduce vulnerabilities, making Python applications less prone to security breaches.

Backward compatibility is a crucial consideration for Python updates, and while Python 3.10 brings many enhancements, it also removes deprecated features to streamline the language. For example, old-style string formatting and certain outdated modules have been removed or updated, encouraging developers to adopt modern programming practices. Python 3.10 also refines the behavior of certain built-in functions, ensuring consistency and better performance across different programming paradigms.

The adoption of Python 3.10 in the developer community has been positive, with many organizations upgrading their projects to take advantage of the new features. It has proven to be a beneficial upgrade for software engineers, data scientists, and researchers who rely on Python for various applications. Python's strong ecosystem, including its vast collection of libraries such as NumPy, Pandas, TensorFlow, and PyTorch, continues to thrive with each new release, making Python 3.10 an attractive option for developers worldwide.

Additionally, Python 3.10 enhances asynchronous programming with refinements to coroutine behavior, making event-driven applications and network programming more efficient. Developers working on web applications, microservices, and distributed systems benefit greatly from these improvements. The improved compatibility with modern CPU architectures and multi-core processing also makes Python 3.10 a suitable choice for large-scale enterprise applications.

Python 3.10 is also significant for educators and new programmers, as its improvements in error messages and code clarity make learning Python easier than ever. The enhanced tracebacks provide better insights into debugging, allowing students and junior developers to understand mistakes more intuitively. The combination of improved performance, clearer syntax, and enhanced security makes Python 3.10 a well-rounded and highly adaptable programming language.

Overall, Python 3.10 is a major milestone in the evolution of Python, offering significant improvements in syntax, performance, error handling, and security. With its continuous advancements, Python remains one of the most popular and versatile programming languages in the world, catering to beginners and experienced developers alike. Whether used for web development, data analysis, artificial intelligence, or automation, Python 3.10 ensures that developers have access to a powerful and efficient toolset for their projects.

3.5 ABOUT OPENCV, MEDIAPIPE, PYAUDIO, FACE RECOGNITION, MATPLOTLIB

OPENCV:

OpenCV (OpenSource Computer Vision Library) is a powerful open-source library designed for real-time computer vision and image processing tasks. It provides a comprehensive collection of optimized algorithms for image analysis, object detection, facial recognition, and video processing. Due to its efficiency and speed, OpenCV is widely used in applications that require real-time performance, such as robotics, surveillance, augmented reality, and autonomous vehicles.

One of the key advantages of OpenCV is its cross-platform support, allowing developers to integrate it seamlessly into various operating systems, including Windows, Linux, macOS, and Android. Additionally, it supports multiple programming languages, including Python, C++, and Java, making it accessible to a broad range of developers and researchers. With its extensive functionality and open-source nature, OpenCV continues to be a fundamental tool in the field of computer vision, enabling advancements in artificial intelligence, machine learning, and automation.

MEDIAPIPE:

MediaPipe is an open-source framework developed by Google that enables efficient and customizable machine learning solutions for real-time computer vision applications. It provides pre-built pipelines for tasks such as face detection, hand tracking, pose estimation, and object tracking, making it a powerful tool for developers working with AI-driven visual applications.

One of MediaPipe's key advantages is its ability to run seamlessly on various platforms, including mobile devices, desktops, and web applications. It supports multiple programming languages such as Python and C++, ensuring flexibility for developers across different domains. Additionally, MediaPipe is optimized for real-time performance, allowing efficient execution even on resource-constrained devices. Its ease of integration and extensive functionalities make it a preferred choice for building interactive AI applications, gesture-based controls, and advanced video analysis system

PYAUDIO:

PyAudio is a Python library that provides an interface for working with audio streams using the PortAudio library. It allows developers to record and play audio in real time, making it a valuable tool for applications such as voice recognition, sound analysis, and audio processing. With its simple and flexible API, PyAudio enables users to capture audio from microphones, process it, and output sound through speakers or other audio devices.

One of the key advantages of PyAudio is its cross-platform support, allowing it to run on Windows, macOS, and Linux. It provides low-latency audio streaming, which is essential for applications that require real-time performance, such as voice assistants and audio-based machine learning models. PyAudio is widely used in speech recognition projects, podcast recording applications, and sound visualization tools. Its ease of integration with other Python libraries, such as NumPy and SciPy, makes it an essential library for developers working with audio data.

FACE RECOGNITION:

Face recognition is a biometric technology that identifies or verifies individuals based on their facial features. It uses advanced image processing and machine learning techniques to detect, analyze, and compare facial patterns from images or video streams. This technology is widely applied in security systems, surveillance, authentication, and social media applications.

Face recognition systems work by extracting key facial landmarks, such as the eyes, nose, and mouth, and mapping them into a unique digital representation. Modern deep learning models, particularly convolutional neural networks (CNNs), have significantly improved accuracy in face recognition, even in challenging conditions such as varying lighting, angles, and expressions. With advancements in artificial intelligence, face recognition continues to enhance security, access control, and personalized user experiences, making it a crucial tool in various industries including law enforcement, banking, and healthcare.

MATPLOTLIB:

Matplotlib is a powerful and widely used data visualization library in Python that enables users to create a variety of static, animated, and interactive plots. It provides an extensive set of tools to generate high-quality graphs, charts, and figures, making it an essential tool for data scientists, engineers, and researchers. With Matplotlib, users can create line plots, bar charts, histograms, scatter plots, and more, allowing for effective data representation and analysis. The library is highly customizable, offering options to modify colors, labels, axes, and legends, making it suitable for professional presentations and publications. One of its key components, **pyplot**, simplifies the process of creating plots by providing a MATLAB-like interface, making it accessible even for beginners. Additionally, Matplotlib integrates seamlessly with other scientific libraries such as NumPy and Pandas, allowing for efficient data manipulation and visualization. It also supports multiple output formats, including PNG, PDF, and SVG, making it versatile for various applications. Whether used for exploratory data analysis, statistical modeling, or machine learning visualization, Matplotlib remains a fundamental tool for anyone working with data in Python.

4.1 SYSTEM FLOW DIAGRAM

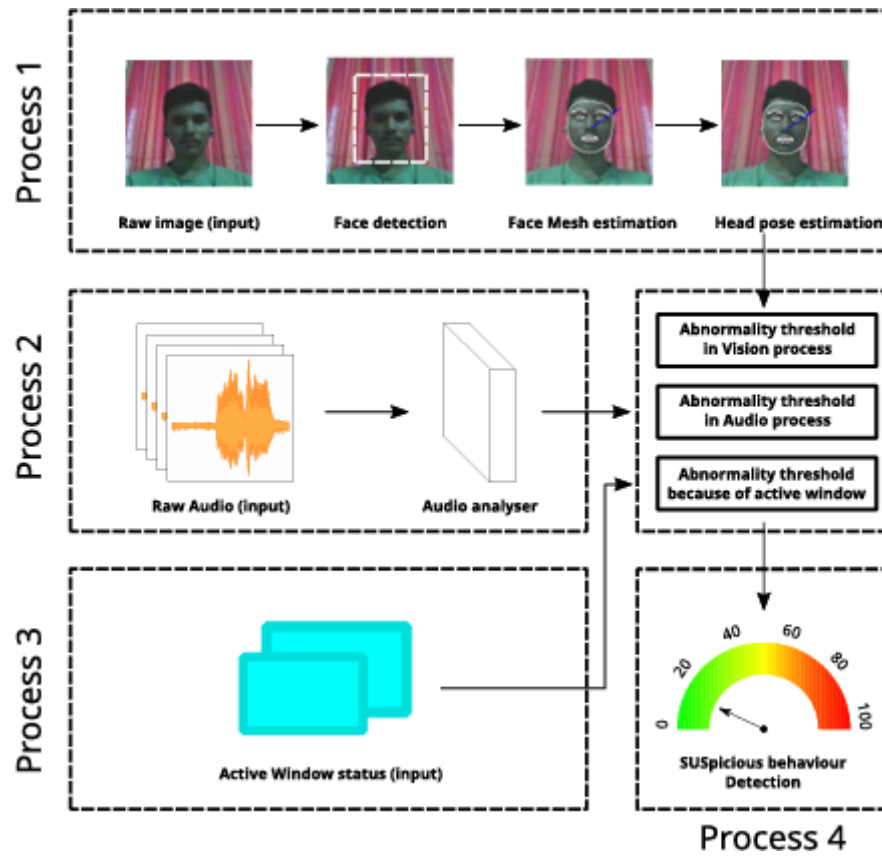


Fig 4.1 System Flow Diagram

4.2 IMPLEMENT METHODOLOGY

Three different methods are used in this proposed system which explain below:

- Head pose estimation
- Speech Detection
- Cheating Behaviour detection

For head pose estimation, we use OpenCV library for python for images capture and preprocessing. The MediaPipe library by Google is used for face recognition. It is an open-source ML library for various computer vision applications. We use the face recognition module to detect the face in the captured image. Then we use the PnP library to calculate the angle of the head and use that data to get the head angle data. We apply a threshold to the angle of the head pose on the X and Y axis such as if the user looks beyond the threshold, then it is detected as looking beyond the screen.

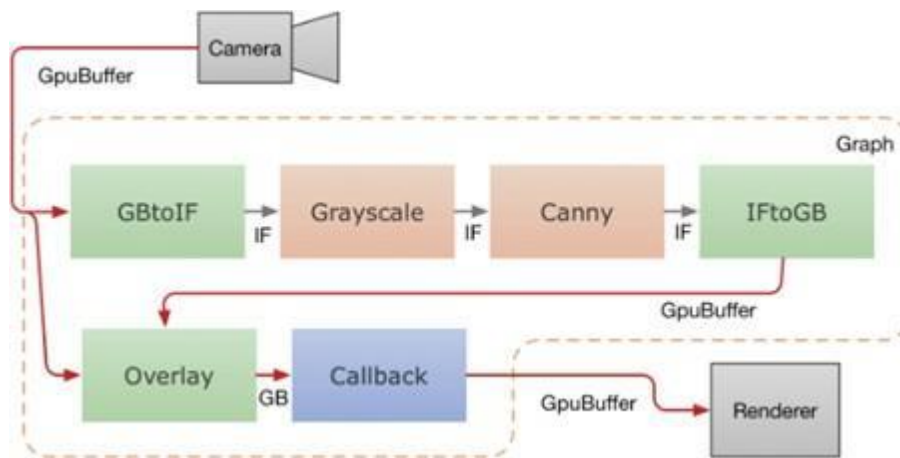


Fig 4.2 Face Recognition Algorithm Processing.

If the user is looking right or left then the value of x-axis changes. If the user is looking left then it goes to the negative side of zero and if the user looks right then the value goes to the positive side of zero. And similarly, if the user looks up or down then the y-axis value changes. The values of x-axis and y-axis are then passed through an algorithm which checks whether it goes above the threshold then it changes the flag.

We detect audio in student's environment using sounddevice library in python. We use frames as input, and get its amplitude. Number of frames is monitored continuously and its amplitude is averaged out. If this average is greater than the threshold set by us over a period of time we consider it as suspicious behaviour and feed appropriate weights to our algorithm for further processing.

For suspicious behaviour detection we have used different flags pertaining to different inputs and these flags are then fed into a conditional algorithm which has set biases for different inputs. When a condition is satisfied, the resulting percentage output is added with a factor to the previous percentage to make the resulting percentage by time graph more continuous and not be in a step graph. When the suspicious percentage is above the set threshold then it is detected as cheating.

Previous Cheat	X axis	Y axis	Audio cheat	Final Cheat Percentage
0	0	0	0	0
0	0	0	1	0.2
0	0	1	0	0.2
0	0	1	1	0.4
0	1	0	0	0.1
0	1	0	1	0.4
0	1	1	0	0.15
0	1	1	1	0.25
1	0	0	0	0
1	0	0	1	0.55
1	0	1	0	0.55
1	0	1	1	0.85
1	1	0	0	0.6
1	1	0	1	0.85
1	1	1	0	0.5
1	1	1	1	0.85

Fig 4.3 Weightages for conditional algorithm.

5. SYSTEM DESIGN AND DEVELOPMENT

The system design and development process involves planning, structuring, and implementing an efficient AI-powered proctoring system for cheat detection. This phase ensures seamless integration between machine learning models, real-time video processing, and data analytics while maintaining accuracy, security, and efficiency.

System Design

System design involves defining the architecture, components, data flow, and interactions within the system. The AI-based proctoring system consists of the following key components:

- **Camera Input:** Captures the real-time video feed of the test taker.
- **Face and Landmark Detection:** Identifies the user's face and tracks key facial landmarks.
- **Head Pose Estimation:** Analyses the user's head movements to detect suspicious behaviour.
- **Cheat Probability Calculation:** Uses machine learning models to determine the likelihood of cheating based on predefined criteria.
- **Graphical Representation:** Displays a real-time graph of cheat probability over time.
- **Alert System:** Notifies the examiner if the probability crosses a threshold.

The system follows a structured workflow:

- The camera captures real-time video of the test taker.
- The face and key facial landmarks are detected using OpenCV and MediaPipe.
- The system analyzes head movements and gaze direction to identify unusual patterns.
- A machine learning model calculates the probability of cheating based on deviations from normal behavior.
- A real-time graph updates the cheat probability over time.
- If the probability exceeds a predefined threshold, an alert is triggered.

System Development

The development process focuses on integrating software components to ensure smooth functionality. Key development aspects include:

- Face Detection and Landmark Tracking: Implemented using OpenCV and MediaPipe to track facial features in real-time.
- Machine Learning for Cheat Detection: Uses TensorFlow models to analyze behavior patterns and predict cheat probability.
- Graphical Representation of Results: Uses Matplotlib to generate real-time graphs displaying cheat probability trends.
- Data Processing and Optimization: Ensures efficient real-time analysis with minimal latency.
- Error Handling and Performance Tuning: Implements techniques to reduce false positives and improve accuracy.
- Security Measures: Prevents unauthorized tampering and ensures data privacy.

The system is designed to be scalable, allowing future enhancements such as voice activity detection, multiple test-taker monitoring, and integration with online examination platforms.

6. SYSTEM TESTING AND MAINTENANCE

The System Testing and Maintenance phase ensures that the AI-based Online Exam Proctoring System functions efficiently, securely, and accurately. This phase involves rigorous testing to validate performance, security, and usability, followed by ongoing maintenance to keep the system updated and reliable.

System Testing

System testing verifies the correct functioning of all components and includes the following testing types:

1. Functional Testing

- Ensures the AI-based proctoring system correctly detects and records user activity.
- Verifies facial recognition and identity authentication processes.
- Confirms that flagged violations (e.g., multiple faces, suspicious movements) are logged correctly.

2. Performance Testing

- Tests system response time during live proctoring.
- Evaluates behavior under multiple concurrent exams to ensure scalability.
- Measures the accuracy of AI-based detection in various lighting and environmental conditions.

3. Security Testing

- Ensures secure user authentication to prevent unauthorized access.
- Verifies encrypted data storage and transmission to protect sensitive information.
- Checks for potential vulnerabilities that could allow cheating or data manipulation.

4. Usability Testing

- Ensures an intuitive user interface for students and examiners.
- Tests the ease of setting up and managing exams through the system.

- Evaluates feedback mechanisms for detected violations (e.g., alerts for suspicious activity).

System Maintenance

Regular maintenance is required to keep the system up-to-date, secure, and efficient. Maintenance activities include:

1. Software Maintenance

- Updating AI models to improve detection accuracy.
- Fixing bugs and enhancing system performance.
- Integrating new features or compatibility updates with learning management systems.

2. Security Maintenance

- Regularly updating encryption and security protocols.
- Monitoring for suspicious activity and unauthorized access attempts.
- Conducting periodic security audits to identify and fix vulnerabilities.

3. Data Management

- Ensuring regular backups of exam records and proctoring logs.
- Removing outdated or duplicate data to maintain storage efficiency.
- Improving data retrieval and reporting for better exam analysis.

7. SAMPLE CODING

Main Code

```
import audio

import head_pose

import detection

import threading as th


if __name__ == "__main__": #
    main()

    head_pose_thread = th.Thread(target=head_pose.pose)
    audio_thread = th.Thread(target=audio.sound)
    detection_thread = th.Thread(target=detection.run_detection)


    head_pose_thread.start()
    audio_thread.start()
    detection_thread.start()


    head_pose_thread.join()
    audio_thread.join()
    detection_thread.join()
```

Audio Processing Code

```
import sounddevice as sd

import numpy as np


# Placeholders and global variables

SOUND_AMPLITUDE = 0

AUDIO_CHEAT = 0


# Sound variables

CALLBACKS_PER_SECOND = 38          # Callbacks per second (system dependent)
SUS_FINDING_FREQUENCY = 2          # Calculates SUS *n* times every second
SOUND_AMPLITUDE_THRESHOLD = 20     # Amplitude considered for SUS calc


# Packing *n* frames to calculate SUS

FRAMES_COUNT = int(CALLBACKS_PER_SECOND / SUS_FINDING_FREQUENCY)
AMPLITUDE_LIST = [0] * FRAMES_COUNT
SUS_COUNT = 0

count = 0


def calculate_rms(indata):

    """Calculate the Root Mean Square (RMS) value of the audio data."""

    return np.sqrt(np.mean(indata**2)) * 1000 # Scaling factor for better readability


def print_sound(indata, outdata, frames, time, status):
    global SOUND_AMPLITUDE, SUS_COUNT, count,
    SOUND_AMPLITUDE_THRESHOLD, AUDIO_CHEAT

    rms_amplitude = calculate_rms(indata)
```

```

AMPLITUDE_LIST.append(rms_amplitude)

count += 1

AMPLITUDE_LIST.pop(0)

if count == FRAMES_COUNT:

    avg_amp = sum(AMPLITUDE_LIST) / FRAMES_COUNT

    SOUND_AMPLITUDE = avg_amp

    if SUS_COUNT >= 2:

        AUDIO_CHEAT = 1

        SUS_COUNT = 0

    if avg_amp > SOUND_AMPLITUDE_THRESHOLD:

        SUS_COUNT += 1

    else:

        SUS_COUNT = 0

        AUDIO_CHEAT = 0

count = 0

def sound():

    with sd.Stream(callback=print_sound):

        sd.sleep(-1)

def sound_analysis():

    global AMPLITUDE_LIST, FRAMES_COUNT, SOUND_AMPLITUDE

    while True:

```

```
AMPLITUDE_LIST.append(SOUND_AMPLITUDE)

AMPLITUDE_LIST.pop(0)

avg_amp = sum(AMPLITUDE_LIST) / FRAMES_COUNT

if avg_amp > 10:
    print("Sus...")

if __name__ == "__main__":
    sound()
```

8. SAMPLE SCREEN DISPLAY

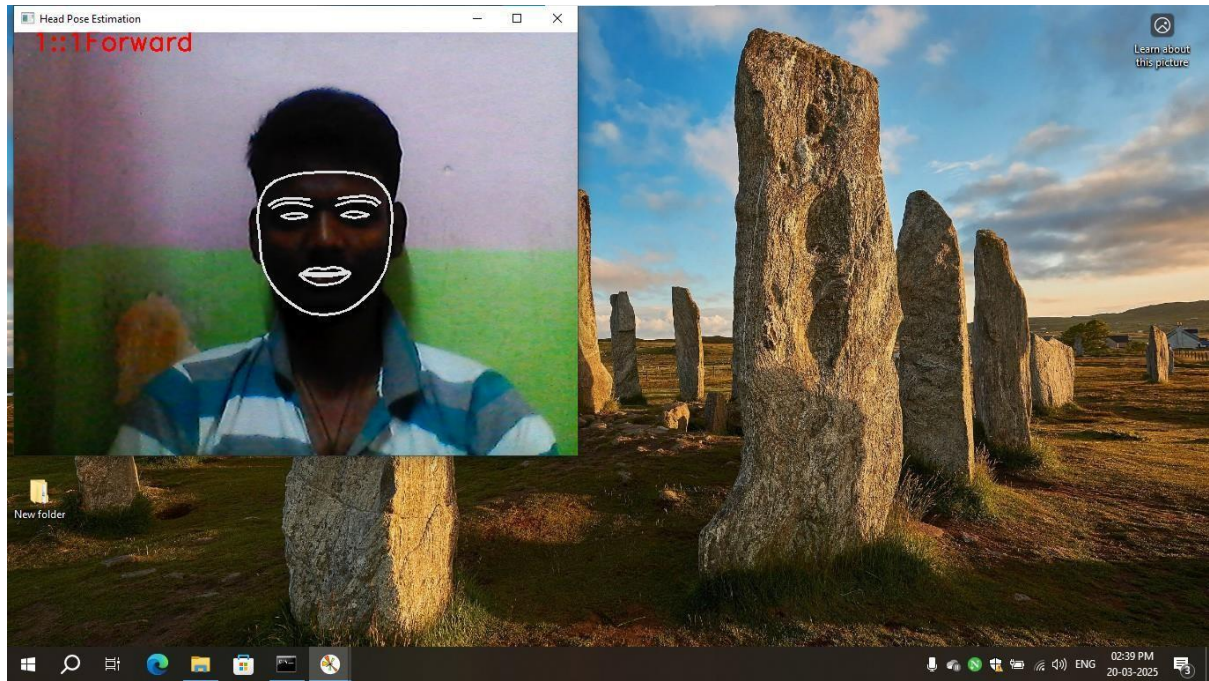


Fig 8.1 Head Pose Estimation for Proctoring

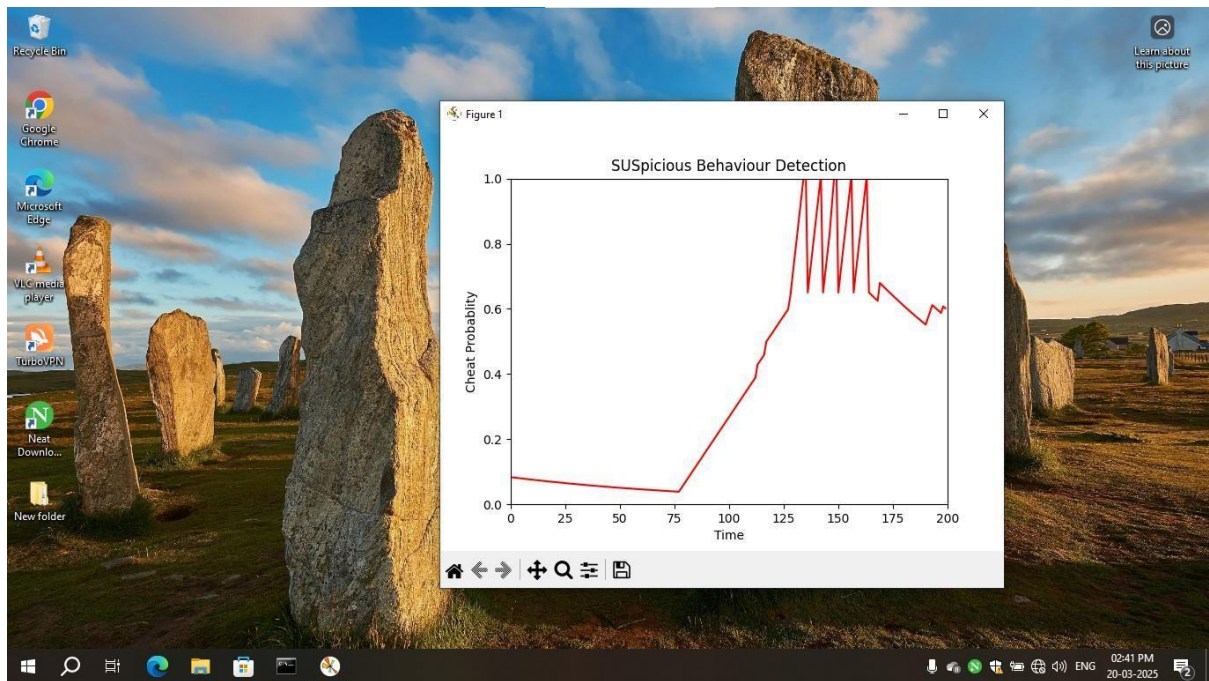


Fig 8.2 Cheating Probability Over Time

```
C:\Windows\System32\cmd.exe

(venv) C:\Users\Prinson Jose\Desktop\New folder\proctoring-main\src>python run.py
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
W0000 00:00:1742288725.846530 776 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feed
back tensors.
W0000 00:00:1742288725.917316 8104 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feed
back tensors.
C:\Users\Prinson Jose\Desktop\New folder\proctoring-main\src\detection.py:87: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
plt.show()
C:\Users\Prinson Jose\Desktop\New folder\proctoring-main\src\detection.py:88: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
axes = plt.gca()
C:\Users\Prinson Jose\Desktop\New folder\proctoring-main\src\detection.py:102: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
plt.pause(1e-17)
Cheat percent: 0.01 0
Cheat percent: 0.009900990099009901 0
Cheat percent: 0.01 0
W0000 00:00:1742288728.652426 776 landmark_projection_calculator.cc:186] Using NORM_RECT without IMAGE_DIMENSIONS is only supported for the square ROI. Provide IMAG
E_DIMENSIONS or use PROJECTION_MATRIX.
Cheat percent: 0.009900990099009901 0
Cheat percent: 0.029900990099009907 0
Cheat percent: 0.04990099009900991 0
Cheat percent: 0.06990099009900991 0
Cheat percent: 0.08990099009900991 0
Cheat percent: 0.10990099009900992 0
Cheat percent: 0.1299009900990099 0
Cheat percent: 0.14990099009900992 0
Cheat percent: 0.16990099009900994 0
Cheat percent: 0.18990099009900996 0
Cheat percent: 0.20990099009900998 0
Cheat percent: 0.22990099009901 0
Cheat percent: 0.24990099009901 0
Cheat percent: 0.26990099009901003 0
Cheat percent: 0.28990099009901005 0
Cheat percent: 0.30990099009901007 0
Cheat percent: 0.3299009900990101 0
Cheat percent: 0.3499009900990101 0
Cheat percent: 0.3699009900990101 0
Cheat percent: 0.38990099009901014 0
Cheat percent: 0.40990099009901015 0
Cheat percent: 0.42990099009901017 0
Cheat percent: 0.4499009900990102 0
Cheat percent: 0.4699009900990102 0
Cheat percent: 0.46524850504852494 0
```

Fig 8.3 Real-Time Cheat Detection Output

9. CONCLUSION

The AI-based Online Exam Proctoring System provides an efficient and automated solution for conducting secure online examinations. By utilizing artificial intelligence, this system ensures real-time monitoring, preventing cheating and unauthorized activities during exams. The system employs facial recognition, eye tracking, and behavior analysis to detect suspicious movements, ensuring the integrity of the examination process.

One of the key advantages of this system is its ability to provide a seamless and scalable solution for remote assessments. Unlike traditional invigilation methods, AI-based proctoring eliminates the need for physical supervisors, making it cost-effective and accessible to a wider audience. Additionally, real-time monitoring and automated flagging of suspicious activities enhance security and accuracy.

The system also incorporates secure authentication mechanisms to verify candidates' identities, reducing the chances of impersonation and fraud. It ensures a fair examination environment by detecting multiple faces, screen switching, or unauthorized device usage. Future enhancements may include advanced AI algorithms for improved accuracy, integration with learning management systems, and cloud-based storage for seamless exam management.

Overall, the AI-based Online Exam Proctoring System is a reliable, scalable, and secure solution for modern online assessments, ensuring fairness, transparency, and convenience for both institutions and students.

10. REFERENCES

- [0] A. W. Muzaffar, M. Tahir, M. W. Anwar, Q. Chaudry, S. R. Mir and Y. Rasheed, "A Systematic Review of Online Exams Solutions in E-Learning: Techniques, Tools, and Global Adoption," in IEEE Access, vol. 9, pp. 32689-32712, 2021, doi: 10.1109/ACCESS.2021.3060192.
- [1] F. Schroff, D. Kalenichenko and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 815-823, doi: 10.1109/CVPR.2015.7298682.
- [2] Abdul Wahid, Yasushi Sengoku, and Masahiro Mambo. 2015. Toward constructing a secure online examination system. In Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication (IMCOM '15). Association for Computing Machinery, New York, NY, USA, Article 95, 1–8. <https://doi.org/10.1145/2701126.2701203>
- [3] Y. Atoum, L. Chen, A. X. Liu, S. D. H. Hsu and X. Liu, "Automated Online Exam Proctoring," in IEEE Transactions on Multimedia, vol. 19, no. 7, pp. 1609-1624, July 2017, doi: 10.1109/TMM.2017.2656064.
- [4] González-González, C.S.; Infante-Moro, A.; Infante-Moro, J.C. Implementation of E-Proctoring in Online Teaching: A Study about Motivational Factors. Sustainability 2020, 12, 3488. <https://doi.org/10.3390/su12083488>